과제 #4

M1522.006700 확장형 고성능 컴퓨팅 (001)

2021-15531 컴퓨터공학부 김인호
November 4, 2025

1 GPU 정보 확인하기

(a) 아래는 각 명령의 실행 결과와 명령의 의미이다. slurm에서 출력하는 job의 queue 대기 및 할당 알림은 출력 결과에서 생략하였다.

```
$ srun --partition=class1 --gres=gpu:4 nvidia-smi

Mon Nov  3 01:16:30 2025
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 520.61.05    Driver Version: 520.61.05    CUDA Version: 11.8     |
|-------------------------------+----------------------+----------------------+
| GPU  Name        Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap|         Memory-Usage | GPU-Util  Compute M. |
|                               |                      |               MIG M. |
|===============================+======================+======================|
|   0  NVIDIA TITAN RTX    On   | 00000000:18:00.0 Off |                  N/A |
| 41%   40C    P8    30W / 280W |      0MiB / 24576MiB |      0%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+
|   1  NVIDIA TITAN RTX    On   | 00000000:3B:00.0 Off |                  N/A |
| 41%   27C    P8    13W / 280W |      0MiB / 24576MiB |      0%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+
|   2  NVIDIA TITAN RTX    On   | 00000000:86:00.0 Off |                  N/A |
| 41%   25C    P8     8W / 280W |      0MiB / 24576MiB |      0%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+
|   3  NVIDIA TITAN RTX    On   | 00000000:AF:00.0 Off |                  N/A |
| 41%   25C    P8     3W / 280W |      0MiB / 24576MiB |      0%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+

+-----------------------------------------------------------------------------+
| Processes:                                                                  |
|  GPU   GI   CI        PID   Type   Process name                  GPU Memory |
|        ID   ID                                                   Usage      |
|=============================================================================|
|  No running processes found                                                 |
+-----------------------------------------------------------------------------+
```

위의 명령은 NVIDIA GPU의 현재 상태를 요약하여 보여주는 `nvidia-smi` 명령을 계산 노드에서

GPU 4개를 할당받아 실행하는 명령이다. GPU의 이름, 온도, 전력 소모, 메모리 사용 등을 시각적으로 보여준다.

```
$ srun --partition=class1 --gres=gpu:4 nvidia-smi -q

==============NVSMI LOG==============

Timestamp                                 : Mon Nov  3 01:24:26 2025
Driver Version                            : 520.61.05
CUDA Version                              : 11.8

Attached GPUs                             : 4
GPU 00000000:18:00.0
    Product Name                          : NVIDIA TITAN RTX
    Product Brand                         : Titan
    Product Architecture                  : Turing
    Display Mode                          : Disabled
    Display Active                        : Disabled
    Persistence Mode                      : Enabled
    MIG Mode
        Current                           : N/A
        Pending                           : N/A
    Accounting Mode                       : Disabled
    Accounting Mode Buffer Size           : 4000
    Driver Model
        Current                           : N/A
        Pending                           : N/A
    Serial Number                         : 1324419051655
    GPU UUID                              : GPU-d7d12c0c-9406-6ae6-beea-8ed0d8d58a71
    Minor Number                          : 0
    VBIOS Version                         : 90.02.2E.00.0C
...(후략)
```

위의 명령은 `nvidia-smi` 명령에 `-q` 옵션을 달아 계산 노드에서 GPU 4개를 할당받아 실행하는 명령이다. 해당 옵션을 달면 GPU의 PCI, 스로틀, 온도, 클럭 등 각종 사양과 현재 상태를 더욱 자세하게 보여준다. 위에서는 생략되었지만, GPU 4개에 대한 자세한 정보가 출력된다.

```
$ srun --partition=class1 --gres=gpu:4 clinfo
Number of platforms                               1
  Platform Name                                   NVIDIA CUDA
  Platform Vendor                                 NVIDIA Corporation
  Platform Version                                OpenCL 3.0 CUDA 11.8.88
  Platform Profile                                FULL_PROFILE
  Platform Extensions                             cl_khr_global_int32_base_atomics cl_khr_global_int32_ext
  Platform Host timer resolution                  0ns
  Platform Extensions function suffix             NV

  Platform Name                                   NVIDIA CUDA
Number of devices                                 4
  Device Name                                     NVIDIA TITAN RTX
  Device Vendor                                   NVIDIA Corporation
  Device Vendor ID                                0x10de
  Device Version                                  OpenCL 3.0 CUDA
  Driver Version                                  520.61.05
```

```
  Device OpenCL C Version                          OpenCL C 1.2
  Device Type                                      GPU
  Device Topology (NV)                             PCI-E, 18:00.0
  Device Profile                                   FULL_PROFILE
...(후략)
```

위 명령은 OpenCL 플랫폼 정보와 그 플랫폼의 장치(GPU) 정보를 보여주는 명령어이다. 플랫폼 이름(NVIDIA CUDA)와 플랫폼 관련 정보, 장치 이름과 정보 등이 출력된다. 위에서는 생략되었지만 장치(GPU) 4개에 대한 정보가 모두 출력된다.

(b) GPU의 모델명: NVIDIA TITAN RTX
노드당 장착된 GPU의 개수: 4

(c) GPU 하나의 메모리 크기: 24576 MiB

(d) GPU의 maximum power limit: 280 W
GPU의 maximum SM clock speed: 1770 MHz

(e) Max work item dimensions: 3
Max work item sizes: 1024x1024x64 (각 차원의 최대 크기)
Max work group size: 1024

2 Matrix Multiplication using OpenCL

- **병렬화 방식**
  $(N \times M)$ 크기의 행렬 C를 정의하기 위해, 커널의 index space를 $M \times N$ 크기로 정의하여 각 work item (i, j)가 C_{ij}를 계산하도록 하였다. 또한, index space를 $\texttt{TS} \times \texttt{TS}$ 크기의 work group들로 나누어 한 work group이 $\texttt{TS} \times \texttt{TS}$ 크기의 행렬 C 일부분을 계산하도록 하였다. 이 work group은 계산해야 하는 데이터를 타일 단위(TS)로 쪼개어서 각 그룹이 실행하는 compute unit의 local memory를 최대한 활용할 수 있도록 하였다. 이를 위해 $C_{i:i+\texttt{TS},j:j+\texttt{TS}}$를 계산하는 데 필요한 총 데이터인 $A_{i:i+\texttt{TS},0:K}, B_{0:K,i:i+\texttt{TS}}$의 각 K차원 부분 또한 타일 단위로 쪼개어 계산한 뒤, 그 계산 결과의 합을 C에 쓰는 것으로 구현하였다. ($A_{n:m,p:q}$는 행렬 A의 n행부터 m행까지의 부분 중 p열부터 q열까지의 부분을 의미)

- **뼈대 코드 `matmul.c` 설명**
  `matmul.c`는 `matmul_initialize` 함수, `matmul` 함수, `matmul_finalize` 함수와 다른 보조 함수들로 이루어져 있다.

  먼저 `matmul_initialize` 함수는 OpenCL을 이용하여 행렬곱 연산을 kernel에서 실행시키기 위해 필요한 여러 요소를 생성하고, 행렬곱의 대상이 되는 행렬을 저장할 메모리 버퍼를 생성하는 역할을 한다. 이 함수에서 사용한 `create_and_build_program_with_source` 함수는 `kernel.cl`의 내용을 읽어 문자열로 저장한 뒤 해당 프로그램을 빌드하는 부분을 맡는다. 이 두 함수에서 사용한 API와 설명은 아래와 같다.

  - `clGetPlatformIDs`: 현재 사용 가능한 OpenCL 플랫폼(의 리스트)을 얻는다.
  - `clGetDeviceIDs`: 인자로 준 OpenCL 플랫폼에서 사용 가능한 장치(의 리스트)를 얻는다.
  - `clCreateContext`: 커널의 실행 환경인 OpenCL 컨텍스트를 생성한다.
  - `clCreateCommandQueue`: 특정 계산 장치(device)에서 실행할 명령들을 담는 명령 큐를 생성한다.
  - `clCreateProgramWithSource`: 컨텍스트와 소스 코드를 받아 OpenCL 프로그램 오브젝트를 생성한다.
  - `clBuildProgram`: 프로그램 오브젝트를 실행할 수 있도록 빌드(컴파일 및 링크)한다.
  - `clCreateKernel`: 프로그램으로부터 특정 함수명에 해당하는 커널 오브젝트를 생성한다.

- `clCreateBuffer`: 버퍼 오브젝트를 만들고 메모리를 할당한다.

`matmul` 함수는 커널 함수의 인자와 입력할 데이터를 설정하고 실제로 실행하는 역할을 한다.

- `clSetKernelArg`: 커널 오브젝트(여기서는 `sgemm` 함수)에 인자로 넣을 값을 설정한다.
- `clEnqueueWriteBuffer`: 호스트 프로그램의 데이터를 버퍼에 쓰는 명령을 큐에 넣는다.
- `clEnqueueNDRangeKernel`: index space를 설정하고, 커널을 실행하는 명령을 큐에 넣는다.
- `clEnqueueReadBuffer`: 버퍼의 데이터를 호스트 프로그램을 읽는 명령을 큐에 넣는다.

`matmul_finalize` 함수는 앞에서 할당했던 모든 OpenCL 자원들을 순서대로 해제한다.

- `clReleaseMemObject`, `clReleaseKernel`, `clReleaseProgram`, `clReleaseCommandQueue`, `clReleaseContext`: 각각 메모리(여기서는 버퍼), 커널, 프로그램, 명령 큐, 컨텍스트 오브젝트를 할당 해제한다.

- **코드 최적화 방식 및 성능 실험**

  현재 kernel 코드에는 타일링(Tiling), 로컬 메모리를 활용한 행렬 로드 등의 최적화 방식이 적용되어 있고, `matmul.c`에도 최적화를 적용한 부분이 있다. 현재의 코드를 가지고 `run_performance.sh`를 실행한 결과, 약 1800 GFLOPS의 성능을 보였다.

  먼저, `kernel.cl`의 루프문에는 인자로 받은 행렬을 로컬 메모리로 옮기는 코드가 포함되어 있다. 원래 함수의 인자는 글로벌 메모리에 저장되어 있는 데이터이므로, 이를 로컬 메모리로 옮김으로써 메모리에 접근하는 시간을 단축할 수 있다. 이 부분을 적용했을 때와, 적용하지 않았을 때의 성능을 측정해보면 각각 1811.50 GFLOPS, 1350.70 GFLOPS로 측정된다. 따라서 이 부분 최적화는 성능을 약 34.1% 향상시킨다고 할 수 있다.

  이 코드에서 가장 중요한 최적화는 타일링이다. 위 '병렬화 방식' 파트에서 서술한 것처럼 `matmul` 함수와 `kernel.cl`에 각각 `TS`라는 변수를 값이 같도록 설정하여, 하나의 타일 안에서 행렬곱 연산이 반복적으로 이루어질 수 있도록 하였다. 계산을 위해 읽어야 할 행렬 A와 B를 쪼개면, 캐시에 데이터가 더 오래 머무르며 메모리에 직접 접근하는 횟수를 줄이므로 속도를 크게 향상시킬 수 있다. 이 `TS`를 1로 설정하게 되면, 타일링을 하지 않은 효과를 볼 수 있다. `TS`의 값을 변화시키며 성능을 측정한 결과는 아래와 같다. OpenCL의 Max work group size가 1024이므로, 최댓값은 32이다.

Tile size(`TS`)	1	2	4	8	16	32
성능(GFLOPS)	8.04	60.04	356.22	1140.72	1709.32	1836.34

* `TS` = 1의 경우는 시간이 지나치게 오래 걸려 1회씩 따로 3번 측정하여 평균을 내었다.